From raw EMG to interpretable features

September 7, 2026

1 From raw EMG to interpretable features

1.0.1 A visual Rock–Paper–Scissors tutorial

Follow one recording through **offset removal → notch filters → band-pass filtering → detrending → normalization → sliding windows → RMS features**. Every stage keeps an intermediate result so that its effect can be inspected.

We use the public EMG-RPS dataset on OSF and the filter settings in this repository. The default is **session E3, player P1**, with both EMG channels and calibration trial `S1_1`. Change the configuration below to explore another recording.

What you will learn - Read labeled EMG recordings and identify a trial. - Compare a signal in time and frequency before and after filtering. - Understand what normalization changes, and what it leaves intact. - Turn 51 samples into a pair of RMS features and inspect their relationship to gesture labels.

This notebook ends at features; it does not train or load a model. Run it from the `emg-rps` root. A fresh checkout can download the OSF archive; existing `data/E*_data.csv` files are reused.

1.1 0 · Setup

Install the packages below.

[24]:
```
!pip install --quiet numpy pandas scipy matplotlib ipython
```

[7]:
```
from pathlib import Path
import shutil
import tempfile
import zipfile
from urllib.request import Request, urlopen
from urllib.error import URLError, HTTPError

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.signal import butter, ellip, sosfiltfilt, sosfreqz, detrend, welch
from numpy.lib.stride_tricks import sliding_window_view
from IPython.display import display
```

[8]:
```
# Change these, then restart the kernel and run all cells.
DATA_DIR = Path("data")
SESSION = 3                    # E1 ... E12
PLAYER = 1                     # P1 or P2; E2/P1 is excluded in the source pipeline
TRIAL = "S1_1"                # calibration annotations: S1_1 ... S1_15
FS = 512                       # nominal samples per second, as in the repository
WINDOW_SIZE = 51               # about 100 ms
STEP_SIZE = 1                  # one sample between adjacent window starts

assert 1 <= SESSION <= 12 and PLAYER in (1, 2)
if SESSION == 2 and PLAYER == 1:
    raise ValueError("E2/P1 is excluded because of a device problem; choose another player/session.")
if not TRIAL.startswith("S1_"):
    raise ValueError("This tutorial uses a calibration trial: choose S1_1 ...S1_15.")

CHANNELS = [f"P{PLAYER}_CH1", f"P{PLAYER}_CH2"]
GESTURES = {1: "Rock", 2: "Paper", 3: "Scissors"}
COLORS = {"before": "#64748b", "after": "#007f86", "accent": "#d6752c",
          "Rock": "#007f86", "Paper": "#bd6035", "Scissors": "#7355a5"}
plt.rcParams.update({
    "figure.figsize": (11, 4), "figure.dpi": 120, "font.size": 10,
    "axes.titlesize": 12, "axes.labelsize": 10, "axes.spines.top": False,
    "axes.spines.right": False, "axes.grid": True, "grid.alpha": 0.18,
    "axes.facecolor": "#fafbfc", "figure.facecolor": "white",
    "savefig.facecolor": "white", "legend.frameon": False,
})
```

1.2 1 · Load the OSF recording, with a local cache

The archive URL below is the one used by the original notebook. OSF may package the dataset inside another ZIP. The loader handles both a direct CSV and a nested archive, and extracts only the selected recording into `data/`.

The first download may contain the entire dataset even though we select one CSV. It is cached as `data/osf_fmrea.zip`. If OSF is unavailable, download the dataset from its project page and place `E3_data.csv` (or your selected session) directly in `data/`, then rerun. Existing CSVs are never downloaded again.

[9]:
```
OSF_ARCHIVE_URL = "https://files.osf.io/v1/resources/fmrea/providers/osfstorage/?zip=1"

def extract_csv_from_archive(archive_path, filename, destination, depth=0):
    """Find one CSV, including inside nested ZIPs, and copy it to a fixed path."""
    if depth > 3:
```

```
        raise FileNotFoundError(f"Archive nesting is too deep to find␣
↪{filename}.")
    destination = Path(destination)
    with zipfile.ZipFile(archive_path) as archive:
        matches = [item for item in archive.infolist()
                   if not item.is_dir() and Path(item.filename).name ==␣
↪filename]
        if len(matches) > 1:
            raise ValueError(f"More than one {filename} found in {archive_path};
↪ extract manually.")
        if matches:
            destination.parent.mkdir(parents=True, exist_ok=True)
            partial = destination.with_suffix(".csv.part")
            try:
                with archive.open(matches[0]) as source, partial.open("wb") as␣
↪target:
                    shutil.copyfileobj(source, target)
                partial.replace(destination)
            finally:
                partial.unlink(missing_ok=True)
            return destination
        for item in archive.infolist():
            if item.filename.lower().endswith(".zip"):
                with tempfile.TemporaryDirectory() as tmp:
                    inner = Path(tmp) / "inner.zip"
                    with archive.open(item) as source, inner.open("wb") as␣
↪target:
                        shutil.copyfileobj(source, target)
                    try:
                        return extract_csv_from_archive(inner, filename,␣
↪destination, depth + 1)
                    except FileNotFoundError:
                        pass
    raise FileNotFoundError(f"{filename} was not found in {archive_path}.")


def ensure_recording(data_dir, session):
    data_dir = Path(data_dir)
    destination = data_dir / f"E{session}_data.csv"
    if destination.is_file():
        print(f"Using local recording: {destination}")
        return destination
    data_dir.mkdir(parents=True, exist_ok=True)
    archive = data_dir / "osf_fmrea.zip"
    if not archive.exists():
        partial = archive.with_suffix(".zip.part")
```

```
        print("Downloading the OSF archive; the first download may take several␣
↪minutes...")
        try:
            request = Request(OSF_ARCHIVE_URL, headers={"User-Agent":␣
↪"EMG-RPS-tutorial/1.0"})
            with urlopen(request, timeout=120) as response, partial.open("wb")␣
↪as target:
                shutil.copyfileobj(response, target)
            if not zipfile.is_zipfile(partial):
                raise ValueError("OSF returned a response that is not a ZIP␣
↪archive.")
            partial.replace(archive)
        except (URLError, HTTPError, TimeoutError, OSError, ValueError) as exc:
            raise RuntimeError(
                f"Could not download OSF data: {exc}. Download from https://osf.
↪io/fmrea/ "
                f"and place {destination.name} in {data_dir.resolve()}, then␣
↪rerun."
            ) from exc
        finally:
            partial.unlink(missing_ok=True)
    if not zipfile.is_zipfile(archive):
        raise ValueError(f"{archive} is not a valid ZIP; replace it with a␣
↪complete download.")
    print(f"Extracting {destination.name} from {archive}")
    return extract_csv_from_archive(archive, destination.name, destination)
```

[10]:
```
csv_path = ensure_recording(DATA_DIR, SESSION)
recording = pd.read_csv(csv_path, low_memory=False)
required = {"timestamp", "annotation", f"P{PLAYER}_action", *CHANNELS}
missing = required - set(recording.columns)
if missing:
    raise ValueError(f"CSV is missing columns: {sorted(missing)}")
raw = recording[CHANNELS].to_numpy(dtype=float)
if not np.isfinite(raw).all():
    raise ValueError("EMG contains missing or non-finite samples; inspect the␣
↪recording before filtering.")

# Use sample position for a uniform nominal 512 Hz time axis.
time_s = np.arange(len(recording)) / FS
annotations = recording["annotation"].fillna("").astype(str)
trial_indices = np.flatnonzero(annotations.eq(TRIAL).to_numpy())
if not len(trial_indices) or not np.all(np.diff(trial_indices) == 1):
    raise ValueError(f"{TRIAL} is missing or is not one contiguous segment.")
trial_start, trial_stop = trial_indices[0], trial_indices[-1] + 1
view = slice(max(0, trial_start - FS), min(len(raw), trial_stop + FS))
```

```
relative_time = time_s - time_s[trial_start]
trial_duration = len(trial_indices) / FS
label = int(recording[f"P{PLAYER}_action"].iloc[trial_start])
print(f"E{SESSION} / P{PLAYER}: {len(recording):,} samples, {time_s[-1]:.1f} s␣
 ↪nominal duration")
print(f"Selected {TRIAL}: {GESTURES.get(label, label)}, {trial_duration:.2f} s")
print(f"Median timestamp increment: {recording.timestamp.diff().median():.4f}␣
 ↪ms; nominal: {1000 / FS:.4f} ms")
display(recording[["timestamp", *CHANNELS, "annotation", f"P{PLAYER}_action"]].
 ↪iloc[trial_start:trial_start+5])
```

```
Using local recording: data/E3_data.csv
E3 / P1: 369,500 samples, 721.7 s nominal duration
Selected S1_1: Scissors, 5.00 s
Median timestamp increment: 1.9531 ms; nominal: 1.9531 ms
```

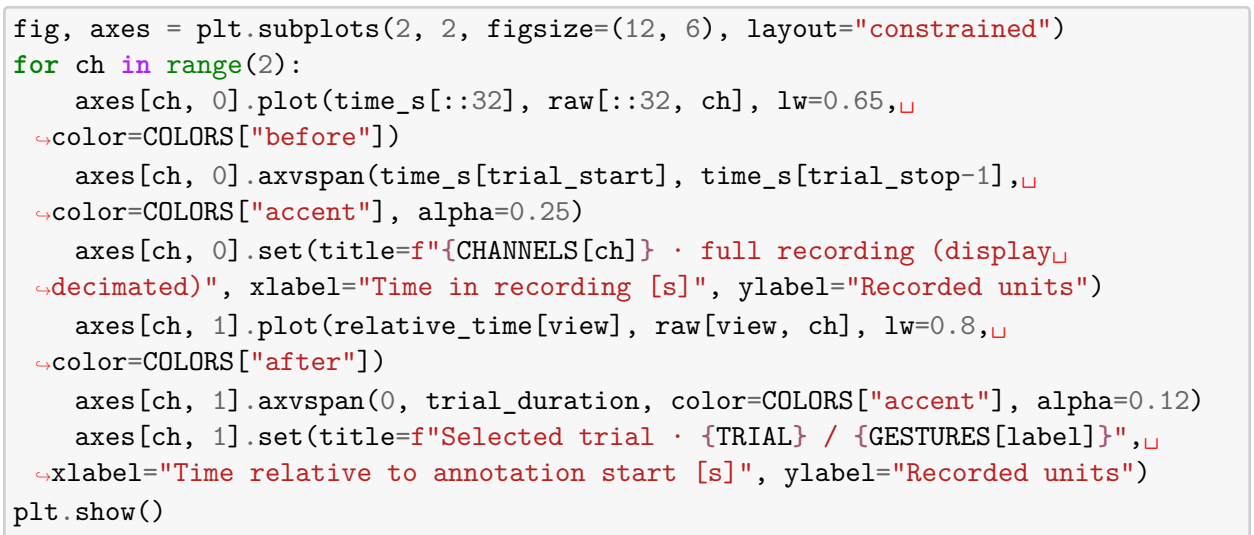
	timestamp	P1_CH1	P1_CH2	annotation	P1_action
93094	1.733295e+12	21.159986	-26.530775	S1_1	3
93095	1.733295e+12	21.159435	-26.529429	S1_1	3
93096	1.733295e+12	21.156409	-26.527697	S1_1	3
93097	1.733295e+12	21.154131	-26.526209	S1_1	3
93098	1.733295e+12	21.152254	-26.530393	S1_1	3

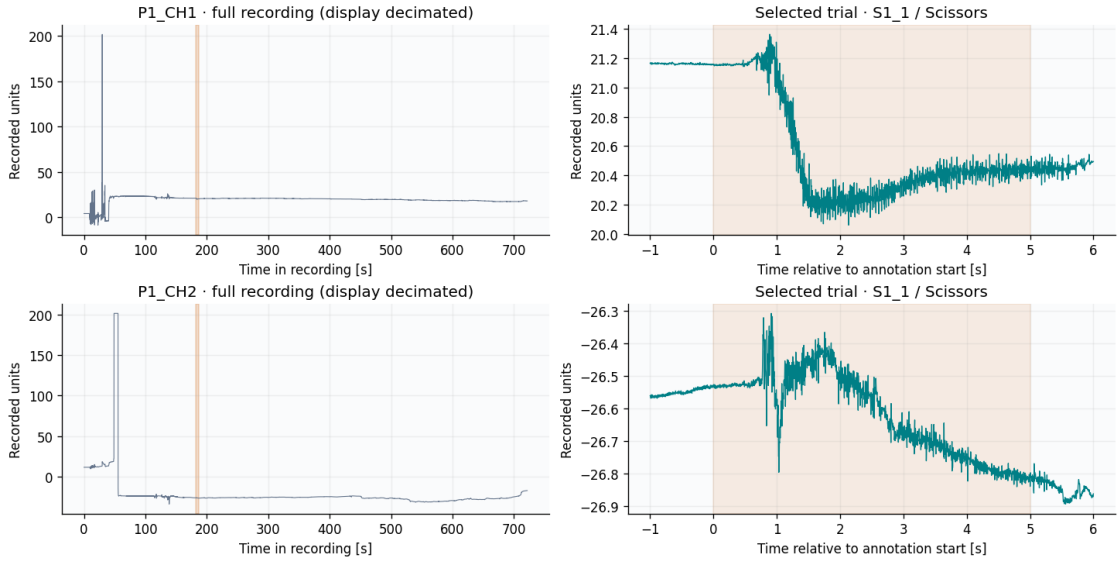
1.2.1 Read the raw signal before changing it

The shaded region is the selected **annotation interval**, not an estimated physiological onset. `P1_CH1` and `P1_CH2` are the two channels for player 1; gesture codes are 1 = rock, 2 = paper, 3 = scissors. The CSV does not establish calibrated amplitude units here, so plots use **recorded units**, not volts.

The overview is drawn with fewer points for readability; filtering always uses every sample. Small timestamp irregularities may exist: this tutorial follows the source pipeline's nominal 512 Hz sample grid and does not resample the data.

[11]:
```
fig, axes = plt.subplots(2, 2, figsize=(12, 6), layout="constrained")
for ch in range(2):
    axes[ch, 0].plot(time_s[::32], raw[::32, ch], lw=0.65,␣
 ↪color=COLORS["before"])
    axes[ch, 0].axvspan(time_s[trial_start], time_s[trial_stop-1],␣
 ↪color=COLORS["accent"], alpha=0.25)
    axes[ch, 0].set(title=f"{CHANNELS[ch]} · full recording (display␣
 ↪decimated)", xlabel="Time in recording [s]", ylabel="Recorded units")
    axes[ch, 1].plot(relative_time[view], raw[view, ch], lw=0.8,␣
 ↪color=COLORS["after"])
    axes[ch, 1].axvspan(0, trial_duration, color=COLORS["accent"], alpha=0.12)
    axes[ch, 1].set(title=f"Selected trial · {TRIAL} / {GESTURES[label]}",␣
 ↪xlabel="Time relative to annotation start [s]", ylabel="Recorded units")
plt.show()
```





1.3 2 · Keep every intermediate preprocessing stage

We use the following filter settings: Butterworth stop filters with `N=4` at **60, 120, 180 and 240 Hz**, each with cutoffs **±3 Hz**; an elliptic **5–250 Hz** band-pass with `N=4` (`rp=0.1 dB`, `rs=40 dB`); then linear detrending. These band-stop/band-pass transformations produce eighth-order one-pass filters; forward–backward application doubles the effective order again.

Here filters use **second-order sections** for numerical stability. Forward–backward filtering removes phase delay, but uses future samples: this is an **offline** tutorial, not a real-time processing recipe. Its effective amplitude response is the square of the one-pass response. See SciPy's filtering documentation.

We filter the **continuous recording first**, then select a trial. This avoids creating a filter boundary at every trial, although the recording's own boundaries can still have transients. The notch frequencies are dataset-specific settings, not a universal prescription for every EMG system.

[12]:
```
NOTCH_HZ = (60, 120, 180, 240)
NOTCH_HALF_WIDTH = 3
BAND_HZ = (5, 250)

def preprocess_emg(raw_signal, fs):
    signal = np.asarray(raw_signal, dtype=float)
    if signal.ndim != 2 or signal.shape[0] < 100 or not np.isfinite(signal).
 ↪all():
        raise ValueError("Expected at least 100 finite samples, shaped␣
 ↪(samples, channels).")
    if BAND_HZ[1] >= fs / 2:
        raise ValueError("The upper cutoff must be below the Nyquist frequency.
 ↪")
```

```
    stages = {"Raw": signal.copy(), "Offset removed": signal - signal[0]}
    current = stages["Offset removed"]
    for notch in NOTCH_HZ:
        sos = butter(4, [notch - NOTCH_HALF_WIDTH, notch + NOTCH_HALF_WIDTH],
                     btype="bandstop", fs=fs, output="sos")
        current = sosfiltfilt(sos, current, axis=0)
        stages[f"Notch {notch} Hz"] = current
    sos = ellip(4, 0.1, 40, BAND_HZ, btype="bandpass", fs=fs, output="sos")
    stages["Band-pass"] = sosfiltfilt(sos, current, axis=0)
    stages["Detrended"] = detrend(stages["Band-pass"], axis=0, type="linear")
    return stages


def normalize_calibration(signal, calibration_mask):
    calibration = signal[calibration_mask]
    if len(calibration) < 2:
        raise ValueError("Need at least two calibration samples.")
    mean = calibration.mean(axis=0)
    std = calibration.std(axis=0, ddof=1)  # matches pandas' sample std in the␣
 ↪source pipeline
    if not np.isfinite(std).all() or np.any(std <= np.finfo(float).eps):
        raise ValueError("A calibration channel has zero or invalid variance.")
    return (signal - mean) / std, mean, std


def window_rms(signal, window_size, step_size):
    signal = np.asarray(signal, dtype=float)
    if signal.ndim != 2 or window_size < 1 or step_size < 1:
        raise ValueError("Use a samples-by-channels array and positive window/
 ↪step sizes.")
    if len(signal) < window_size:
        return (np.empty((0, window_size, signal.shape[1])),
                np.empty((0, signal.shape[1])), np.array([], dtype=int))
    windows = sliding_window_view(signal, window_size, axis=0)[::step_size].
 ↪transpose(0, 2, 1)
    starts = np.arange(0, len(signal) - window_size + 1, step_size)
    rms = np.sqrt(np.mean(windows ** 2, axis=1))
    return windows, rms, starts
```

[13]:
```
stages = preprocess_emg(raw, FS)
summary = pd.DataFrame({
    name: {"CH1 mean": values[:, 0].mean(), "CH1 std": values[:, 0].std(),
           "CH2 mean": values[:, 1].mean(), "CH2 std": values[:, 1].std()}
    for name, values in stages.items()
}).T
display(summary.round(5))
```

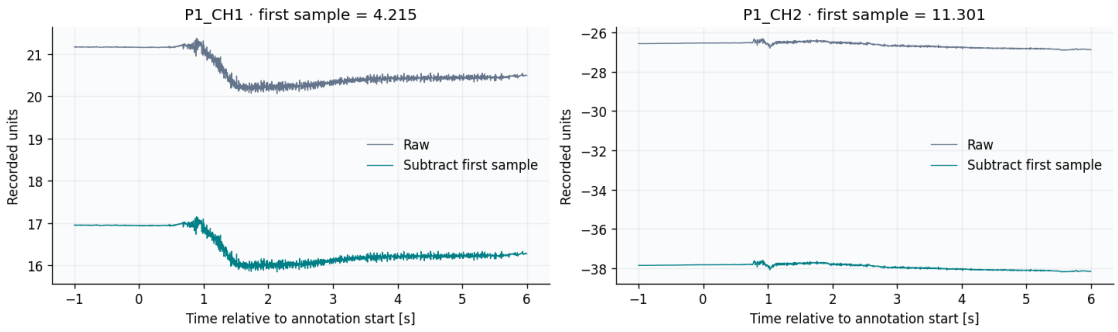
	CH1 mean	CH1 std	CH2 mean	CH2 std
Raw	19.37575	4.92253	-21.69855	23.86535
Offset removed	15.16079	4.92253	-32.99941	23.86535
Notch 60 Hz	15.16079	4.92131	-32.99941	23.86514
Notch 120 Hz	15.16079	4.92123	-32.99941	23.86510
Notch 180 Hz	15.16079	4.92121	-32.99941	23.86510
Notch 240 Hz	15.16079	4.92120	-32.99941	23.86509
Band-pass	0.00152	1.07151	-0.00330	0.99247
Detrended	0.00000	1.07151	-0.00000	0.99247

1.4 3 · Remove the initial offset

Subtract the first sample from each channel: $x_0[n] = x[n] - x[0]$. This **shifts the vertical origin** without changing waveform shape. It does not generally make the whole recording's mean zero, and it is not a noise filter, but it helps avoid filter swinging at the beginning of the signal.

Look for: identical fluctuations at a different vertical position. The high-pass part (applied in step 5) part of the band-pass filter also suppresses DC.

[14]:
```
fig, axes = plt.subplots(1, 2, figsize=(12, 3.5), layout="constrained")
for ch, ax in enumerate(axes):
    ax.plot(relative_time[view], stages["Raw"][view, ch],␣
 ↪color=COLORS["before"], lw=0.8, label="Raw")
    ax.plot(relative_time[view], stages["Offset removed"][view, ch],␣
 ↪color=COLORS["after"], lw=0.8, label="Subtract first sample")
    ax.set(title=f"{CHANNELS[ch]} · first sample = {raw[0, ch]:.3f}",␣
 ↪xlabel="Time relative to annotation start [s]", ylabel="Recorded units")
    ax.legend()
plt.show()
```

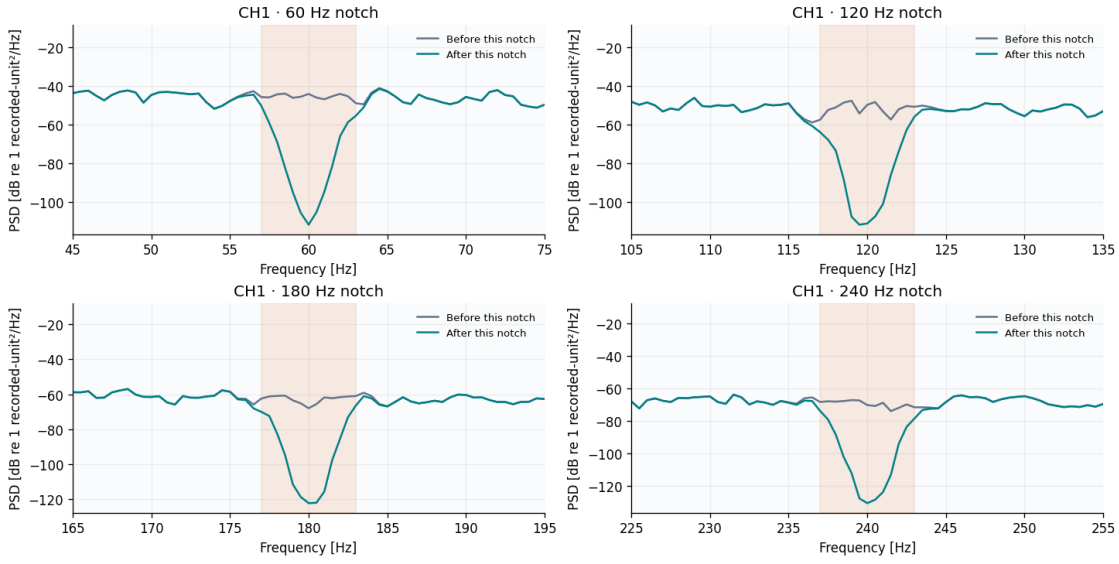


1.5 4 · Remove narrow frequency bands with notch filters

A waveform can hide narrow-band contamination. A **power spectral density (PSD)** plot shows how signal power is distributed over frequency. We use Welch's method, averaging overlapping spectral estimates.

Each panel compares the signal **immediately before and after one notch**. The shaded band marks the filter cutoffs. A dip proves attenuation in that band; it does not prove all removed energy was interference—EMG can also contain energy there. Some recordings may have little contamination to begin with.

[15]:
```
fig, axes = plt.subplots(2, 2, figsize=(12, 6), layout="constrained")
previous = "Offset removed"
for notch, ax in zip(NOTCH_HZ, axes.flat):
    current = f"Notch {notch} Hz"
    for name, color, label_text in [(previous, COLORS["before"], "Before this␣
 ↪notch"),
                                    (current, COLORS["after"], "After this␣
 ↪notch")]:
        frequencies, power = welch(stages[name][trial_indices, 0], fs=FS,␣
 ↪nperseg=1024)
        ax.plot(frequencies, 10 * np.log10(np.maximum(power, 1e-20)),␣
 ↪color=color, lw=1.5, label=label_text)
    ax.axvspan(notch-3, notch+3, color=COLORS["accent"], alpha=0.13)
    ax.set(xlim=(notch-15, min(notch+15, FS/2)), title=f"CH1 · {notch} Hz␣
 ↪notch",
           xlabel="Frequency [Hz]", ylabel="PSD [dB re 1 recorded-unit²/Hz]")
    ax.legend(fontsize=8)
    previous = current
plt.show()
```



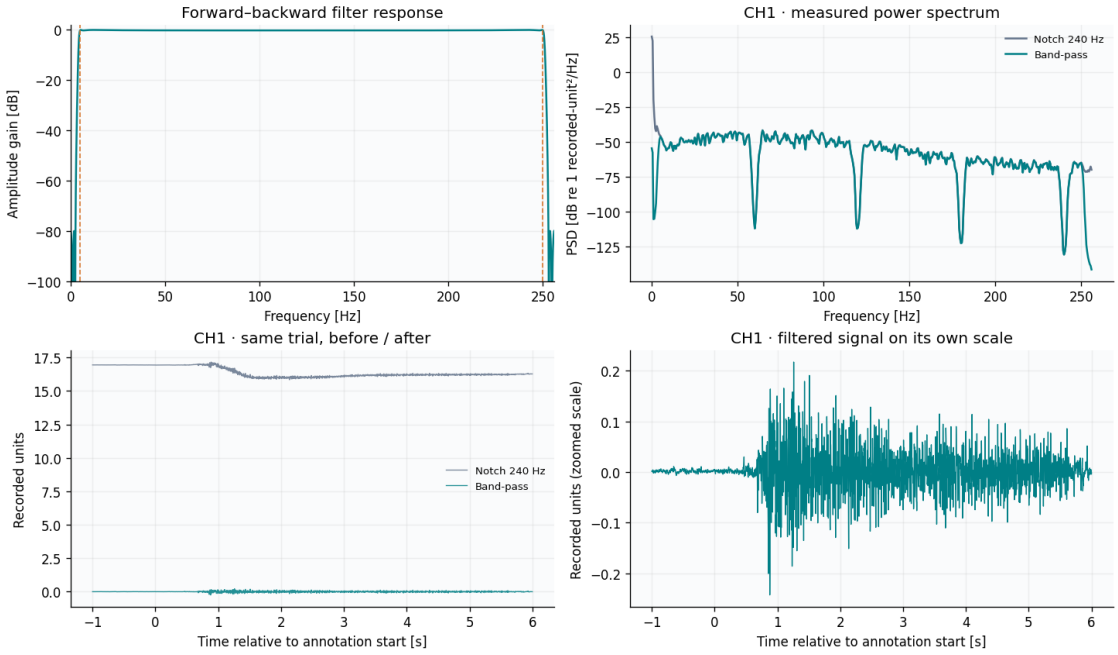
1.6 5 · Keep the 5–250 Hz band

The band-pass attenuates low-frequency drift and frequencies near the 256 Hz Nyquist limit. The response curve describes the filter; the PSD describes this particular recording. Cutoffs are transition-

band boundaries rather than a perfect brick wall.

Look for: changes below 5 Hz and near 250 Hz. Their visual size depends on what was present before filtering. The lower-left panel uses the same amplitude scale for the before/after traces. The lower-right panel zooms the filtered signal so its smaller fluctuations remain visible.

[16]:
```
band_sos = ellip(4, 0.1, 40, BAND_HZ, btype="bandpass", fs=FS, output="sos")
freq, response = sosfreqz(band_sos, worN=4096, fs=FS)
fig, grid = plt.subplots(2, 2, figsize=(12, 7), layout="constrained")
axes = grid.ravel()
axes[0].plot(freq, 20*np.log10(np.maximum(np.abs(response)**2, 1e-6)),␣
 ↪color=COLORS["after"])
axes[0].set(xlim=(0, 256), ylim=(-100, 2), title="Forward-backward filter␣
 ↪response", xlabel="Frequency [Hz]", ylabel="Amplitude gain [dB]")
for cutoff in BAND_HZ:
    axes[0].axvline(cutoff, color=COLORS["accent"], ls="--", lw=1)
for name, color in [("Notch 240 Hz", COLORS["before"]), ("Band-pass",␣
 ↪COLORS["after"])]:
    f, p = welch(stages[name][trial_indices, 0], fs=FS, nperseg=1024,␣
 ↪detrend=False)
    axes[1].plot(f, 10*np.log10(np.maximum(p, 1e-20)), label=name, color=color)
    axes[2].plot(relative_time[view], stages[name][view, 0], label=name,␣
 ↪color=color, lw=0.8, alpha=0.85)
axes[1].set(title="CH1 · measured power spectrum", xlabel="Frequency [Hz]",␣
 ↪ylabel="PSD [dB re 1 recorded-unit²/Hz]")
axes[2].set(title="CH1 · same trial, before / after", xlabel="Time relative to␣
 ↪annotation start [s]", ylabel="Recorded units")
axes[1].legend(fontsize=8)
axes[2].legend(fontsize=8)
axes[3].plot(relative_time[view], stages["Band-pass"][view, 0],␣
 ↪color=COLORS["after"], lw=0.8)
axes[3].set(title="CH1 · filtered signal on its own scale", xlabel="Time␣
 ↪relative to annotation start [s]", ylabel="Recorded units (zoomed scale)")
plt.show()
```

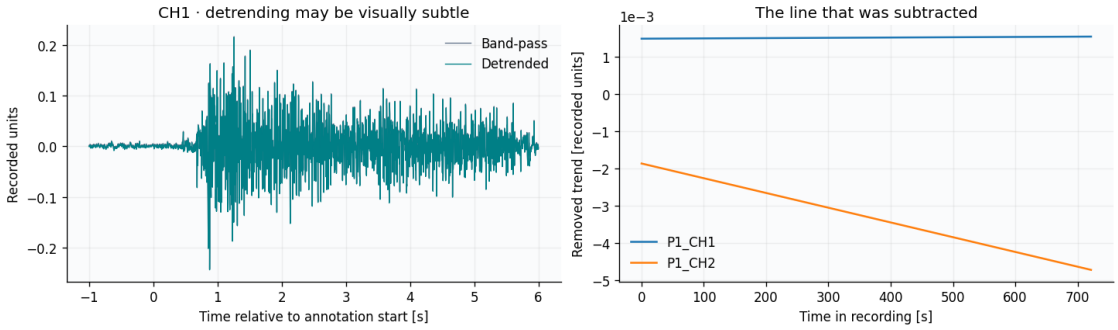


1.7 6 · Remove the remaining linear trend

`detrend` subtracts a least-squares straight line from the full filtered recording. The band-pass has already removed much of the slow variation, so this correction may be tiny. The right panel isolates the removed line rather than exaggerating the before/after difference.

Look for: a signal centered close to zero over the full recording, not necessarily within every trial.

[26]:
```
trend = stages["Band-pass"] - stages["Detrended"]
fig, axes = plt.subplots(1, 2, figsize=(12, 3.5), layout="constrained")
for name, color in [("Band-pass", COLORS["before"]), ("Detrended",␣
 ↪COLORS["after"])]:
    axes[0].plot(relative_time[view], stages[name][view, 0], color=color, lw=0.
 ↪8, label=name)
axes[0].set(title="CH1 · detrending may be visually subtle", xlabel="Time␣
 ↪relative to annotation start [s]", ylabel="Recorded units")
axes[0].legend()
for ch in range(2):
    axes[1].plot(time_s[::32], trend[::32, ch], label=CHANNELS[ch])
axes[1].set(title="The line that was subtracted", xlabel="Time in recording␣
 ↪[s]", ylabel="Removed trend [recorded units]")
axes[1].ticklabel_format(axis="y", style="sci", scilimits=(-2, 2))
axes[1].legend()
plt.show()
```



1.8 7 · Normalize using the calibration portion

For each channel, compute mean μ and sample standard deviation s from valid **S1 calibration samples**, then apply $z[n] = (x[n] - \mu)/s$ to the entire recording. Normalization changes offset and scale, not temporal structure. Its output is dimensionless.

This mirrors the calibration-to-spontaneous normalization strategy in the repository. E4 calibration trial 6 is excluded for both players. If you later evaluate a model, fit normalization only on the chosen training partition; this tutorial does not create such a train/test split.

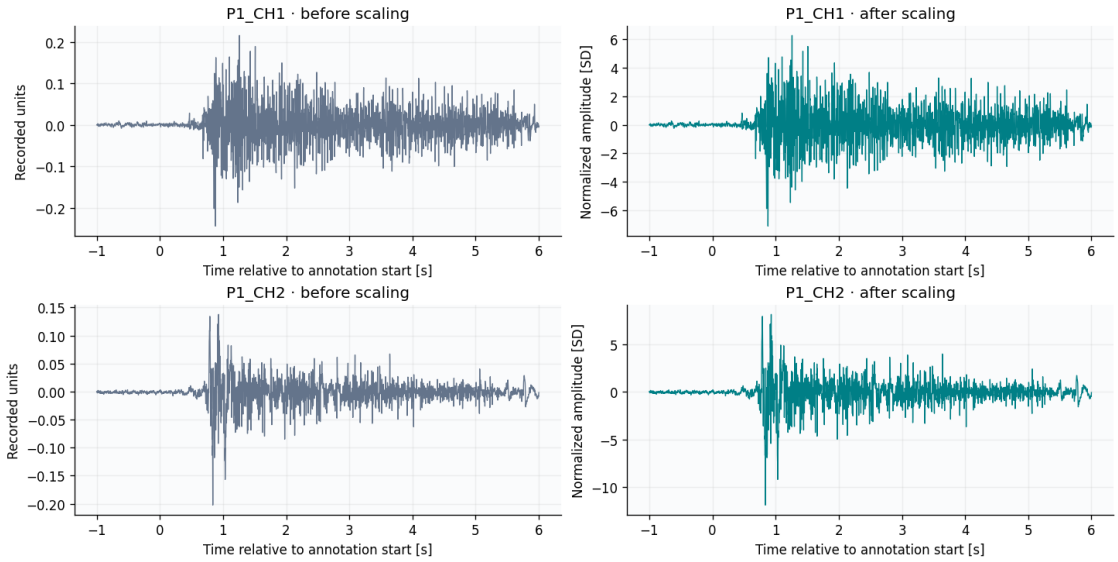
[18]:
```
valid_calibration = [f"S1_{i}" for i in range(1, 16) if not (SESSION == 4 and i␣
 ↪== 6)]
calibration_mask = annotations.isin(valid_calibration).to_numpy()
if TRIAL not in valid_calibration:
    raise ValueError(f"{TRIAL} is excluded for this session; choose another␣
 ↪calibration trial.")
normalized, calibration_mean, calibration_std =␣
 ↪normalize_calibration(stages["Detrended"], calibration_mask)
display(pd.DataFrame({"channel": CHANNELS, "calibration mean (input units)":␣
 ↪calibration_mean,
                      "calibration std (input units)": calibration_std,
                      "normalized mean": normalized[calibration_mask].
 ↪mean(axis=0),
                      "normalized sample std": normalized[calibration_mask].
 ↪std(axis=0, ddof=1)}))

fig, axes = plt.subplots(2, 2, figsize=(12, 6), layout="constrained")
for ch in range(2):
    axes[ch, 0].plot(relative_time[view], stages["Detrended"][view, ch],␣
 ↪color=COLORS["before"], lw=0.8)
    axes[ch, 0].set(title=f"{CHANNELS[ch]} · before scaling", ylabel="Recorded␣
 ↪units", xlabel="Time relative to annotation start [s]")
    axes[ch, 1].plot(relative_time[view], normalized[view, ch],␣
 ↪color=COLORS["after"], lw=0.8)
```

```
    axes[ch, 1].set(title=f"{CHANNELS[ch]} · after scaling", ylabel="Normalized␣
  ↪amplitude [SD]", xlabel="Time relative to annotation start [s]")
plt.show()
```

```
  channel  calibration mean (input units)  calibration std (input units)  \
0  P1_CH1                        0.000148                       0.034458
1  P1_CH2                       -0.000795                       0.016976

   normalized mean  normalized sample std
0    3.550669e-17                    1.0
1    1.801744e-16                    1.0
```



1.9 8 · Turn a short window into one RMS feature

With the default $W = 51$ samples and $f_s = 512$ Hz, the nominal window duration is **99.6 ms**. A stride of one sample advances the window by **1.95 ms**, so neighboring windows share 50 of their 51 samples.

For one channel, the root mean square is

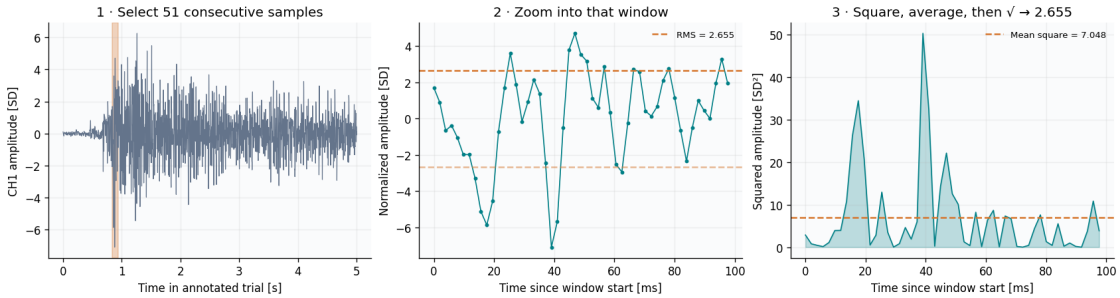
$$\mathrm{RMS}(x) = \sqrt{\frac{1}{W}\sum_{k=0}^{W-1} x[k]^2}.$$

Square → average → square root. Squaring prevents positive and negative amplitudes from cancelling. RMS summarizes magnitude; it is neither an onset detector nor a gesture classifier. For a zero-mean window it equals the population standard deviation, but an individual window need not have zero mean.

The highlighted example is a high-RMS window from the selected trial, chosen so the calculation is easy to see. In plots, each feature is placed at the **center of its window**. That placement does not make the offline filtering causal.

[19]:
```
trial_signal = normalized[trial_indices]
windows, rms, window_starts = window_rms(trial_signal, WINDOW_SIZE, STEP_SIZE)
if not len(windows):
    raise ValueError("The selected trial is shorter than the chosen window.")
centers_s = (window_starts + (WINDOW_SIZE - 1)/2) / FS
example = int(np.argmax(rms[:, 0]))
example_start = window_starts[example]
window_values = windows[example, :, 0]
window_time_ms = np.arange(WINDOW_SIZE) / FS * 1000
mean_square = np.mean(window_values**2)
example_rms = np.sqrt(mean_square)

fig, axes = plt.subplots(1, 3, figsize=(14, 3.7), layout="constrained")
axes[0].plot(np.arange(len(trial_signal))/FS, trial_signal[:, 0], lw=0.7,␣
 ↪color=COLORS["before"])
axes[0].axvspan(example_start/FS, (example_start+WINDOW_SIZE)/FS,␣
 ↪color=COLORS["accent"], alpha=0.3)
axes[0].set(title=f"1 · Select {WINDOW_SIZE} consecutive samples", xlabel="Time␣
 ↪in annotated trial [s]", ylabel="CH1 amplitude [SD]")
axes[1].plot(window_time_ms, window_values, "o-", ms=2.5, lw=1,␣
 ↪color=COLORS["after"])
axes[1].axhline(example_rms, color=COLORS["accent"], ls="--", label=f"RMS =␣
 ↪{example_rms:.3f}")
axes[1].axhline(-example_rms, color=COLORS["accent"], ls="--", alpha=0.5)
axes[1].set(title="2 · Zoom into that window", xlabel="Time since window start␣
 ↪[ms]", ylabel="Normalized amplitude [SD]")
axes[1].legend(fontsize=8)
axes[2].fill_between(window_time_ms, window_values**2, color=COLORS["after"],␣
 ↪alpha=0.25)
axes[2].plot(window_time_ms, window_values**2, color=COLORS["after"], lw=1)
axes[2].axhline(mean_square, color=COLORS["accent"], ls="--", label=f"Mean␣
 ↪square = {mean_square:.3f}")
axes[2].set(title=f"3 · Square, average, then √ → {example_rms:.3f}",␣
 ↪xlabel="Time since window start [ms]", ylabel="Squared amplitude [SD²]")
axes[2].legend(fontsize=8)
plt.show()
print(f"{len(trial_signal):,} samples → {len(windows):,} windows; shape =␣
 ↪{windows.shape} (windows, samples, channels)")
print(f"RMS feature matrix shape: {rms.shape}; one row contains CH1 and CH2 RMS.
 ↪")
```



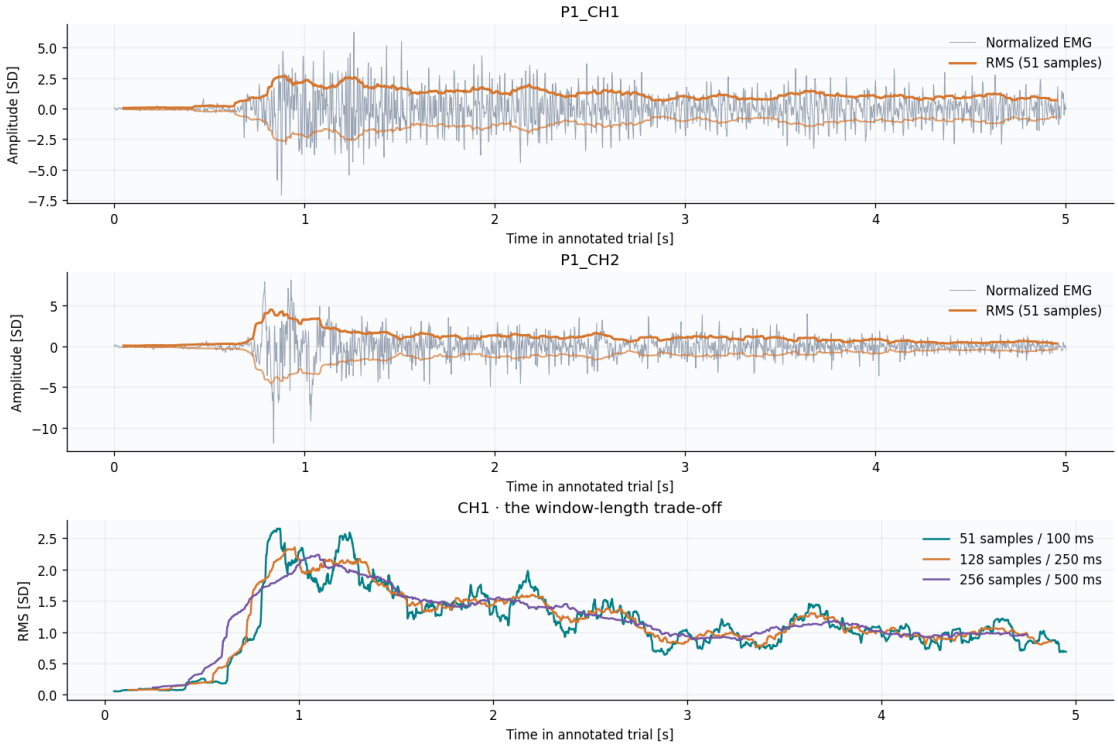

```
2,561 samples → 2,511 windows; shape = (2511, 51, 2) (windows, samples,
channels)
RMS feature matrix shape: (2511, 2); one row contains CH1 and CH2 RMS.
```

1.10 9 · Follow RMS over time, and change the window length

The upper panels overlay **±RMS** on the signed signal. RMS is always nonnegative; the negative copy is drawn only to help compare its magnitude with both sides of the waveform. It is an amplitude summary, not a strict peak envelope.

The lower panel varies the window duration. Longer windows average over more activity and can blur short events; shorter windows follow changes more closely but fluctuate more. No window extends beyond the selected trial, so longer windows begin and end farther from the boundaries.

[20]:
```
fig, axes = plt.subplots(3, 1, figsize=(12, 8), layout="constrained")
for ch in range(2):
    axes[ch].plot(np.arange(len(trial_signal))/FS, trial_signal[:, ch], lw=0.6,␣
 ↪color=COLORS["before"], alpha=0.65, label="Normalized EMG")
    axes[ch].plot(centers_s, rms[:, ch], color=COLORS["accent"], lw=1.8,␣
 ↪label=f"RMS ({WINDOW_SIZE} samples)")
    axes[ch].plot(centers_s, -rms[:, ch], color=COLORS["accent"], lw=1.2,␣
 ↪alpha=0.7)
    axes[ch].set(title=CHANNELS[ch], ylabel="Amplitude [SD]", xlabel="Time in␣
 ↪annotated trial [s]")
    axes[ch].legend(loc="upper right")
for size, color in zip([51, 128, 256], ["#007f86", "#d6752c", "#7355a5"]):
    _, values, starts = window_rms(trial_signal, size, STEP_SIZE)
    axes[2].plot((starts+(size-1)/2)/FS, values[:, 0], color=color, lw=1.5,␣
 ↪label=f"{size} samples / {1000*size/FS:.0f} ms")
axes[2].set(title="CH1 · the window-length trade-off", xlabel="Time in␣
 ↪annotated trial [s]", ylabel="RMS [SD]")
axes[2].legend()
plt.show()
```



1.11 10 · Build a labeled feature table without crossing trial boundaries

Now repeat the same extraction **separately for each valid calibration trial**. Each feature row carries its trial and gesture label. We never form a window by joining the end of one trial to the beginning of the next.

The original scripts can also retain all 51 normalized samples from each channel alongside RMS: that is **102 waveform values + 2 RMS values** per window. Here `windows` lets you inspect the waveform representation, while the compact table below keeps the two RMS features plus metadata for teaching. No feature files are written automatically.

[21]:
```
feature_frames = []
for annotation in valid_calibration:
    indices = np.flatnonzero(annotations.eq(annotation).to_numpy())
    if not len(indices):
        continue
    if not np.all(np.diff(indices) == 1):
        raise ValueError(f"{annotation} is not contiguous; do not join␣
↪separated samples into a window.")
    trial_labels = recording[f"P{PLAYER}_action"].iloc[indices].unique()
    if len(trial_labels) != 1 or trial_labels[0] not in GESTURES:
        raise ValueError(f"Unexpected or changing gesture label inside␣
↪{annotation}.")
```

```
    _, trial_rms, starts = window_rms(normalized[indices], WINDOW_SIZE,␣
 ↪STEP_SIZE)
    feature_frames.append(pd.DataFrame({
        "session": SESSION, "player": PLAYER, "trial": annotation,
        "gesture": GESTURES[int(trial_labels[0])],
        "window_start_sample": indices[starts],
        "window_center_s": (starts + (WINDOW_SIZE-1)/2)/FS,
        "rms_ch1": trial_rms[:, 0], "rms_ch2": trial_rms[:, 1],
    }))
features = pd.concat(feature_frames, ignore_index=True)
display(features.head(8))
display(features.groupby("gesture").agg(trials=("trial", "nunique"),␣
 ↪windows=("trial", "size"),
                                        mean_rms_ch1=("rms_ch1", "mean"),␣
 ↪mean_rms_ch2=("rms_ch2", "mean")))
print(f"Feature table: {features.shape[0]:,} windows × {features.shape[1]}␣
 ↪columns (two features plus metadata).")
```

	session	player	trial	gesture	window_start_sample	window_center_s	\
0	3	1	S1_1	Scissors	93094	0.048828	
1	3	1	S1_1	Scissors	93095	0.050781	
2	3	1	S1_1	Scissors	93096	0.052734	
3	3	1	S1_1	Scissors	93097	0.054688	
4	3	1	S1_1	Scissors	93098	0.056641	
5	3	1	S1_1	Scissors	93099	0.058594	
6	3	1	S1_1	Scissors	93100	0.060547	
7	3	1	S1_1	Scissors	93101	0.062500	

	rms_ch1	rms_ch2
0	0.055282	0.090828
1	0.054633	0.091053
2	0.053739	0.092722
3	0.053769	0.093061
4	0.054112	0.087462
5	0.054209	0.087019
6	0.055383	0.089091
7	0.056050	0.089467

gesture	trials	windows	mean_rms_ch1	mean_rms_ch2
Paper	5	12565	1.036598	1.172590
Rock	5	12563	0.688257	0.498213
Scissors	5	12566	1.045834	0.815004

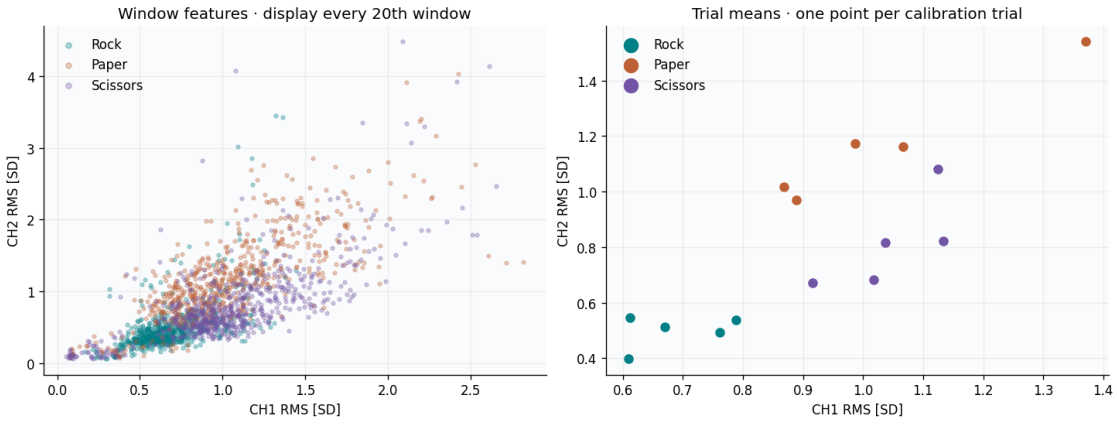
```
Feature table: 37,694 windows × 8 columns (two features plus metadata).
```

1.11.1 Visualize the two-feature representation

Each point represents a window: CH1 RMS on the horizontal axis and CH2 RMS on the vertical axis. The plot displays every twentieth window within each trial to reduce overplotting; the table still contains all windows. Large markers show **per-trial means**, a useful comparison when windows are strongly dependent.

Look for: different locations or overlapping regions for rock, paper and scissors. These are observations about one player's calibration recording. Separation here does **not** establish classification accuracy or generalization to another session or player. Adjacent overlapping windows are not independent observations.

[22]:
```
shown = features.loc[features.groupby("trial", sort=False).cumcount() % 20 == 0]
trial_means = features.groupby(["trial", "gesture"],␣
 ↪as_index=False)[["rms_ch1", "rms_ch2"]].mean()
fig, axes = plt.subplots(1, 2, figsize=(12, 4.5), layout="constrained")
for gesture in GESTURES.values():
    subset = shown[shown.gesture == gesture]
    centers = trial_means[trial_means.gesture == gesture]
    axes[0].scatter(subset.rms_ch1, subset.rms_ch2, s=8, alpha=0.3,␣
 ↪color=COLORS[gesture], label=gesture)
    axes[1].scatter(centers.rms_ch1, centers.rms_ch2, s=70, edgecolor="white",␣
 ↪linewidth=0.8, color=COLORS[gesture], label=gesture)
for ax in axes:
    ax.set(xlabel="CH1 RMS [SD]", ylabel="CH2 RMS [SD]")
    ax.legend(markerscale=1.5)
axes[0].set_title("Window features · display every 20th window")
axes[1].set_title("Trial means · one point per calibration trial")
plt.show()
```



Try next 1. Change `TRIAL` to another S1 annotation. Do the spectra and RMS magnitudes change? 2. Change `PLAYER` or `SESSION`. Does the same gesture occupy the same feature region? 3. Change `WINDOW_SIZE` to 128 and rerun. Which short events become less visible? 4. Change `STEP_SIZE` to 25. The per-window calculation is unchanged, but how many rows remain?

Keep the stages distinct: filtering changes frequency content; detrending removes a fitted line; normalization changes scale; RMS reduces a waveform window to a magnitude summary.